DOCUMENTACION
TECNICA • v2 CORREGIDA

Hangman Project

Juego del Ahorcado en consola — Python.
Documentacion verificada contra el codigo real entregado.

Campo	Detalle
Repositorio	github.[STRIPPED 68 bytes]
Archivos verificados	`main.py, game.py, utils.py, wordlist.txt, README.md, PRESENTACION.md`
Lenguaje / Entorno	Python 3 • Consola • Solo biblioteca estandar (`random, pathlib`)
Como se ejecuta	`python3 main.py o python main.py desde hangman_project/`
Regla central	Adivinar la palabra letra a letra antes de acumular **6 errores** (`MAX_ERRORS = 6`).
Fecha	9 de noviembre de 2026 • v2 corrige la v1 con el codigo real

Que corrige esta v2 — leer antes de entregar

La v1 se hizo solo con el README de GitHub y describia una interfaz *plausible* pero no exacta. Esta v2 se verifico ejecutando el codigo real. Correcciones clave: **(1)** el atributo es `guessed_letters: list`, no `set`, y el contador es `errors`, no `wrong_guesses`; **(2)** `is_over` es **@property**, se usa sin parentesis (`game.is_over`); **(3)** `main()` usa `process_guess()`, no `guess()`; `guess()` existe pero **no la usa main**; **(4)** en `utils.py` las funciones reales son `get_random_word(), is_valid_guess(), display_word(), format_guessed_letters()` — no existen `choose_word(), normalize(), load_wordlist()` ni `format_display()`; **(5)** `display_word()` existe pero **no la usa main**; main usa `display_progress()` que **imprime**, no devuelve; **(6)** las pantallas reales son bilingues y `display_status()` solo aparece **despues** de una jugada nueva y valida; **(7)** `wordlist.txt` tiene exactamente **15 palabras**, listadas en la Seccion 2. Todo lo demas de la v1 (diagramas, casos de uso, algoritmos) se reescribio con estos nombres exactos.

Contenido: 0 Correcciones v1→v2 • 1 Introduccion • 2 Vision general y wordlist real • 3 Arquitectura y diagrama de clases • 4 Diccionario exacto • 5 Casos de uso • 6 Diagramas dinamicos • 7 Simulaciones de pantallas reales • 8 Algoritmos con codigo real • 9 Datos e invariantes • 10 Pruebas verificadas por ejecucion • 11 Limitaciones, riesgos y mejoras • Anexo: presentacion de 5 minutos y checklist

0 Que cambio de la v1 a la v2

Tema	v1 (README solamente)	v2 (codigo real) — correcto
Estado	`guessed: set, wrong_guesses`	`guessed_letters: list[str], errors: int`
Fin de partida	`is_over()` metodo	`is_over` @property, sin ()
Jugada en main	`guess()`	`process_guess()` + chequeo previo de repetida en main
Utils	`choose_word, normalize, load_wordlist, format_display`	`get_random_word, is_valid_guess, display_word, format_guessed_letters`
Mostrar progreso	`display_word() -> str`	`display_progress() -> None (print)` en game; `display_word()` en utils existe pero no se usa
Pantallas	Display antes de la primera jugada, horca ASCII, 'Jugar otra vez'	Sin display inicial, sin horca, sin 'jugar otra vez'; mensajes exactos en ingles + final en espanol
Wordlist	Tamano desconocido	15 palabras, promedio 8.0 letras, todas minusculas a-z

Si ya entregaste la v1, reemplaza el archivo por esta v2. No mezcles ambas: los nombres de la v1 no existen en el codigo.

1 Introduccion

El **Hangman Project** es el juego del ahorcado en consola. El jugador adivina una palabra secreta, elegida al azar, proponiendo letras. Cada acierto revela todas las posiciones de esa letra. Cada error nuevo acerca a perder. Se gana completando la palabra y se pierde al llegar a **6 errores**.

El valor academico, segun `PRESENTACION.md`, esta en aplicar **funciones, clases, modulos, validacion de entradas y lectura de archivos** — no en la dificultad del juego. Esta documentacion demuestra que esa separacion existe de verdad en el codigo: `main.py` es el punto de entrada, `game.py` mantiene estado y reglas, `utils.py` reune validacion, azar y formato, y `wordlist.txt` es el dato.

Alcance

En alcance	Fuera de alcance (codigo actual)
Un jugador, una palabra por partida, consola. Validacion, control de repetidas, 6 errores. Soporte en codigo para caracteres no alfabeticos (espacios / signos se revelan solos).	Interfaz grafica, red, multijugador, puntajes persistentes. Dibujo ASCII de horca, dificultad, pistas, 'jugar otra vez'. Palabras con numeros o tildes en `wordlist.txt` actual (no las hay).

2 Vision general, estructura y wordlist real

Archivo	Rol real (docstring / codigo)	Si cambia esto...
main.py	'Run the Hangman game from the command line.' Bucle + `display_status()`.	... cambia la experiencia en consola, no las reglas.
game.py	'Define the state and rules.' Clase `HangmanGame` + constante `MAX_ERRORS = 6`.	... cambia como se gana / pierde.
utils.py	'Provide input validation and display helpers.' Azar, validacion y formato.	... cambia validacion / formato, sin tocar el estado.
wordlist.txt	15 palabras candidatas, una por linea.	... cambia el vocabulario, sin tocar codigo.
README.md	Descripcion + `python3 main.py` + lista de modulos.	Documentacion.
PRESENTACION.md	Guion de 5 minutos, demo, preguntas probables.	Guia de exposicion (ver Anexo).

Wordlist real — 15 palabras verificadas

Archivo leido tal cual. Todas en minusculas, solo a-z, sin tildes, sin espacios, sin numeros. Largo promedio **8.0** letras. La mas corta es `tiger` (5) y la mas larga `polymorphism` (12). Cada una tiene probabilidad **1/15** ≈ **6.67%** en cada partida.

#	Palabra	Largo	Letras distintas	#	Palabra	Largo	Letras distintas
1	elephant	8	7	9	computer	8	8
2	giraffe	7	5	10	keyboard	8	8
3	penguin	7	6	11	adventure	9	7
4	dolphin	7	7	12	python	6	6
5	tiger	5	5	13	inheritance	11	8
6	butterfly	9	7	14	polymorphism	12	9
7	mountain	8	6	15	geometry	8	7
8	rainbow	7	6				

Consecuencia para la demo: con palabras de 5 a 12 letras, una partida necesita entre 5 y 12 aciertos distintos para ganar, y como maximo 6 errores. El peor caso teorico (adivinar las 26 letras) nunca se alcanza en la practica porque el juego termina antes.

Ejecucion y requisitos

```
$ cd hangman_project
$ python3 main.py     # README
# o
$ python main.py      # PRESENTACION.md

Welcome to Hangman!
Guess a letter:
```

- **Python 3**, sin dependencias externas. Solo `random` y `pathlib.Path` de la biblioteca estandar.
- `get_random_word()` resuelve la ruta con `Path(__file__).with_name('wordlist.txt')`: funciona aunque se ejecute desde otra carpeta, siempre que `wordlist.txt` este junto a `utils.py`.

3 Arquitectura y diagrama de clases

La dependencia es en una sola direccion y esta escrita en los `import` reales: `main.py` importa `HangmanGame` de `game.py` y tres funciones de `utils.py`; `game.py` importa `is_valid_guess` de `utils.py`; `utils.py` no importa a ninguno de los otros dos. Por eso `game.py` y `utils.py` se pueden probar sin consola.

```
main.py  ■■ usa ■■>  HangmanGame (game.py)
   ■                    ■
   ■■■ usa ■■> utils.py ■■ lee ■■> wordlist.txt
   (diagrama ASCII de PRESENTACION.md, confirmado por los imports)
```

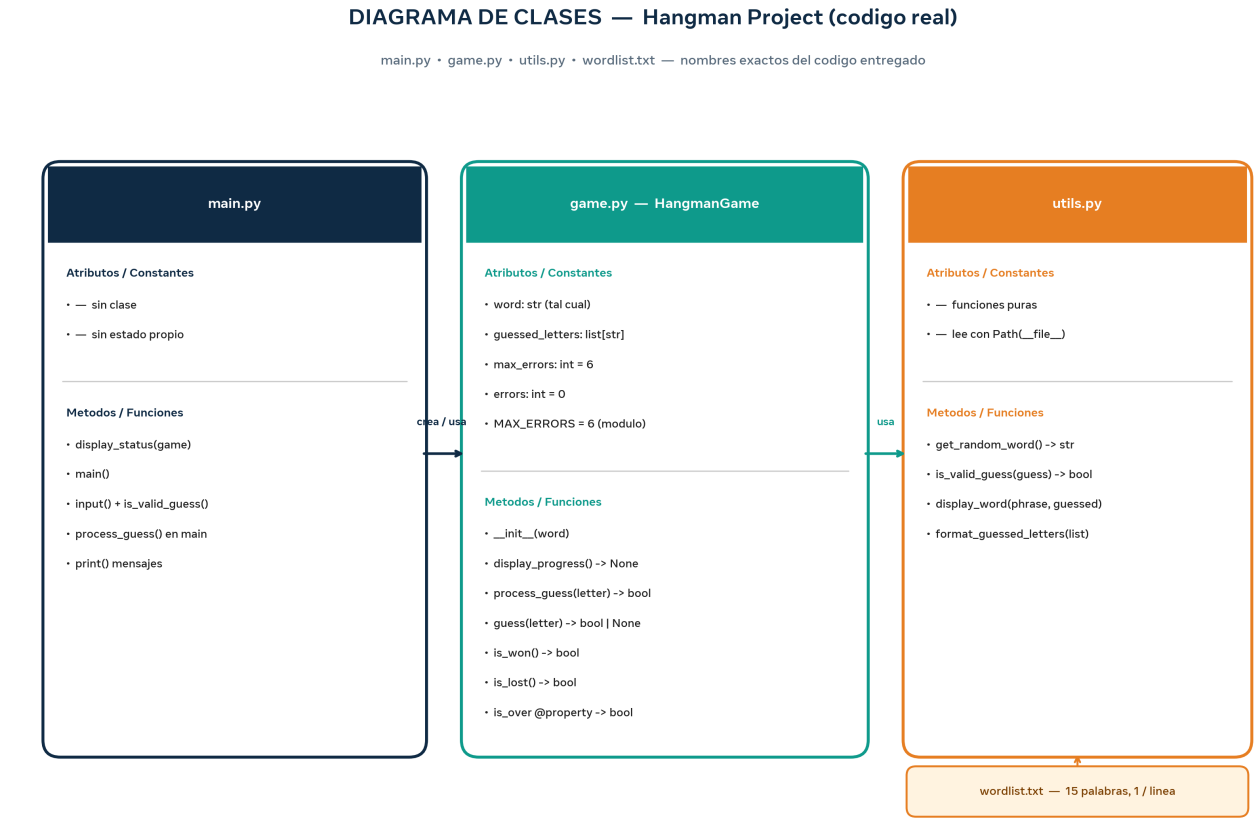


Figura 1 — Diagrama de clases con nombres exactos. Ojo: display_word() y guess() existen pero main no las usa (ver Seccion 4.4).

Lectura

- **main → game:** crea `HangmanGame(get_random_word())`, consulta `is_over` (propiedad), `is_won()`, `guessed_letters`, `errors`, `max_errors`, llama a `process_guess()` y a `display_progress()` via `display_status()`.
- **main → utils:** `get_random_word()`, `is_valid_guess()`, `format_guessed_letters()`.
- **game → utils:** `is_valid_guess()` dentro de `process_guess()`.
- **utils → wordlist:** `get_random_word()` lee el archivo con `Path.read_text(encoding='utf-8')`.

4 Diccionario exacto de clases, modulos y funciones

Cada fila cita el comportamiento real, verificado por ejecucion. Los mensajes de error entre comillas son literales del codigo.

4.1 Constante y clase HangmanGame (game.py)

Elemento	Tipo / Valor	Significado real
`MAX_ERRORS`	int = 6 (modulo)	Constante de modulo. `__init__` la copia a `self.max_errors`. Cambiar la dificultad = cambiar aqui.
`word`	str	Palabra **tal cual** se paso al constructor. **No** se pasa a minusculas en `__init__`. Se compara con `.lower()` en cada metodo. Se muestra tal cual al final.
`guessed_letters`	list[str]	Lista en **orden de jugada**. Es **lista por requisito del ejercicio** (PRESENTACION.md), no set. Buscar en ella es O(k), k ≤ 26.
`max_errors`	int = 6	Copia de `MAX_ERRORS` por instancia.
`errors`	int = 0	Errores acumulados. Solo sube en `process_guess()` con letra nueva e incorrecta. Nunca baja.

Metodo	Firma real	Comportamiento verificado
`__init__`	`__init__(self, word)`	Si `not word` o ningun caracter es alfabetico → `ValueError('word must contain at least one letter')`. Si no, `word=word, guessed_letters=[], max_errors=6, errors=0`.
`display_progress`	`() -> None`	**Imprime**, no devuelve. Por cada caracter: si no es alfabetico → lo muestra tal cual; si su minuscula esta en `guessed_letters` → muestra el caracter original; si no → `'_'`. Imprime `' '.join(progress)`. Ej: `'Python' + ['p']` → `'P _ _ _ _ _'` (conserva la mayuscula original).
`process_guess`	`(letter) -> bool`	Si `not is_valid_guess(letter)` → `ValueError('guess must be a single letter')`. Si no, `letter = letter.lower()`. Si ya estaba → devuelve `letter in word.lower()` **sin** cambiar estado. Si es nueva → la agrega; si no esta en la palabra → `errors += 1`, devuelve False; si esta → devuelve True.
`guess`	`(letter) -> bool | None`	**Diferencia clave:** si la letra (normalizada) ya estaba → devuelve `None` (preserva None como resultado de repetida). Si no → delega en `process_guess(letter)`. **main no usa este metodo.** Es API alternativa para quien use la clase directamente.
`is_won`	`() -> bool`	`all(not isalpha() or lower in guessed_letters for ch in word)`. Los caracteres no alfabeticos **no** hay que adivinarlos.
`is_lost`	`() -> bool`	`errors >= max_errors`.
`is_over`	`@property -> bool`	`is_won() or is_lost()`. **Se accede sin parentesis:** `game.is_over`. Escribir `game.is_over()` daria `TypeError: 'bool' object is not callable`.

Invariantes reales

- 0 ≤ `errors` ≤ `max_errors` (6), mientras el juego se usa via `main` o `process_guess`.
- `guessed_letters` no tiene duplicados: tanto `main` como `process_guess`/`guess` comprueban antes de agregar.
- Repetir una letra nunca cambia `errors` ni `guessed_letters`.
- Si `is_won()` es True, `is_lost()` es False, porque el bucle se detiene al llegar a cualquiera de los dos.

4.2 Modulo main.py

Funcion	Que hace exactamente
`display_status(game)`	Imprime 3 lineas: `'Word:'`, luego `game.display_progress()`, luego `'Guessed letters: {format_guessed_letters(...)}'`, luego `'Attempts remaining: {max_errors - errors}'`. **Solo se llama despues de una jugada nueva y valida.**
`main()`	1) `game = HangmanGame(get_random_word())`. 2) `print('Welcome to Hangman!')`. 3) `while not game.is_over:` pide `input('Guess a letter: ').lower()`, valida, controla repetida, llama a `process_guess()`, si devuelve False imprime `'That letter is not in the phrase.'`, luego `display_status(game)`. 4) Al salir: si `is_won()` → `'¡Ganaste! La palabra era: {word}'`; si no → `'Fin del juego. La palabra era: {word}'`.
`if __name__ == '__main__': main()`	Punto de entrada. Permite importar `main.py` sin ejecutar el juego.

Doble defensa contra repetidas — por que hay dos chequeos

En `main()`: `if guess in game.guessed_letters: print('You already guessed that letter.'); continue` — asi la repetida ni siquiera llega a `process_guess()`. Y en `process_guess()`: si la letra ya estaba, devuelve si es acierto sin cambiar estado. Y en `guess()`: devuelve `None` para repetidas. Son tres capas para el mismo caso, en tres niveles distintos (presentacion, regla principal, API alternativa). En la practica de `main`, la primera capa es la que se ejecuta; las otras dos son defensivas para quien use `HangmanGame` directamente.

4.3 Modulo utils.py — funciones exactas

Funcion	Firma / Codigo clave	Comportamiento verificado
`get_random_word`	`() -> str` (sin parametros)	`Path(__file__).with_name('wordlist.txt').read_text(encoding='utf-8').splitlines()`, filtra `line.strip()` con `.isalpha()`. Si queda vacia → `ValueError('wordlist.txt must contain at least one alphabetic word')`. Si no → `random.choice(words)`. Cada palabra 1/15.
`is_valid_guess`	`(guess) -> bool`	`len(guess) == 1 and guess.isalpha()`. **No** hace `strip()` ni `lower()`: `' A '` (3 caracteres) es invalido, `'A'` es valido, `''`, `'ab'`, `'1'`, `'-'` son invalidos. En `main` el `.lower()` se hace antes, en el `input`.
`display_word`	`(phrase, guessed_letters) -> str`	Devuelve `''.join(...)` **sin espacios**: caracter no alfabetico o adivinado → tal cual; si no → `'_'`. Ej verificado: `display_word('python', ['p','o']) == 'p___o_'`. **No la usa main.** Es una version pura / testeable de la misma idea que `display_progress()` (que imprime con espacios).
`format_guessed_letters`	`(guessed_letters) -> str`	`', '.join(sorted(guessed_letters)) or 'none'`. Orden alfabetico, no orden de jugada. Ej: `['z','a','m']` → `'a, m, z'`. Si la lista esta vacia → `'none'` (en `main` nunca se ve porque `display_status` solo se llama tras una jugada).

4.4 Funciones que existen pero main no usa — no borrar sin entender

Dos funciones estan definidas y son correctas, pero el flujo actual no las llama. Documentarlas como *no usadas* es mas honesto que borrarlas o que fingir que se usan:

- `HangmanGame.guess()`: API alternativa que devuelve `None` para repetidas. `main` usa `process_guess()` + chequeo previo. Si se quiere unificar, `main` podria usar `guess()` y tratar `None` como 'repetida', pero eso cambiaria los mensajes actuales.
- `utils.display_word()`: version pura que devuelve una cadena sin espacios. `main` usa `display_progress()` que imprime con espacios. `display_word()` es util para pruebas (no necesita

capturar `print`), pero hoy no esta integrada.

4.5 Datos — wordlist.txt

Texto plano, una palabra por linea, 15 lineas, sin linea vacia al final que importe (el filtro la descartaria). `get_random_word()` descarta cualquier linea que no sea `.isalpha()` despues de `strip()`, asi que una linea con espacios o numeros no romperia el juego, simplemente se ignoraria.

5 Casos de uso (con mensajes reales)

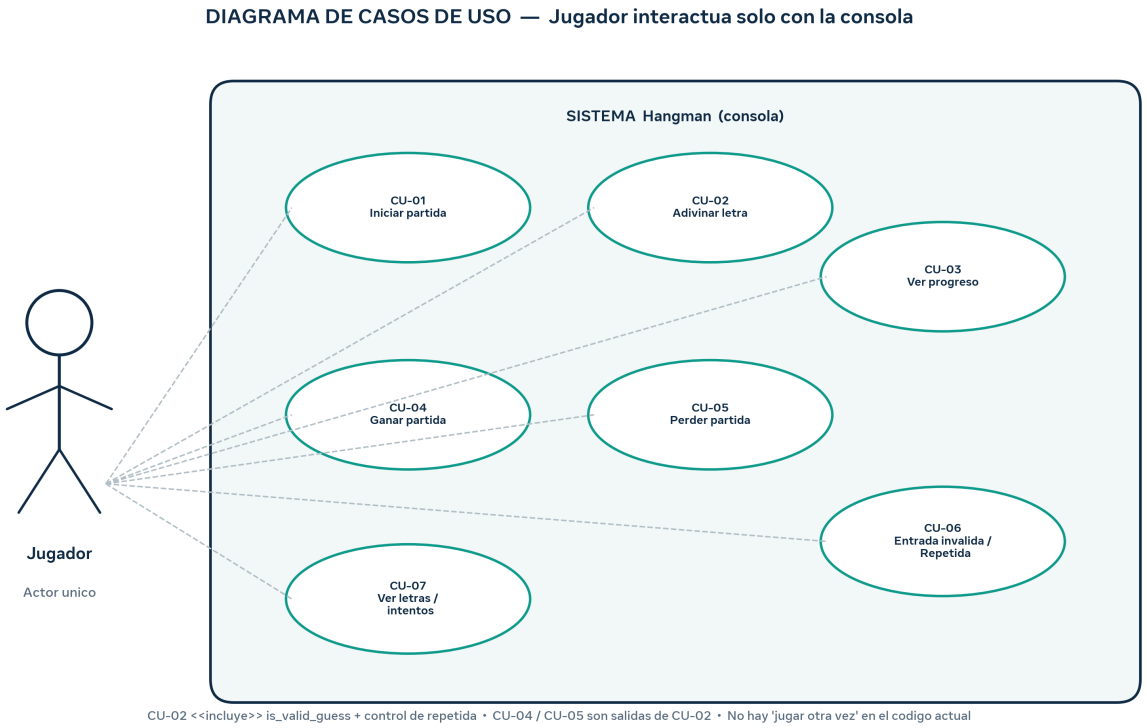


Figura 2 — Casos de uso. No hay 'jugar otra vez' en el codigo actual: CU-06 es entrada invalida / repetida, no reintento de partida.

Especificacion

CU-01 — Iniciar partida

	Jugador ejecuta `python3 main.py`.
	Sistema crea `HangmanGame(get_random_word())` e imprime `'Welcome to Hangman!'` y pide `'Guess a letter: '`. **No** muestra la palabra todavia.
Alternativo / Excepcion	Si `wordlist.txt` esta vacio o sin palabras alfabeticas → `ValueError` y el programa termina con traceback (ver Seccion 11).

CU-02 — Adivinar letra (principal)

	Jugador escribe una letra.
	Sistema hace `.lower()`, valida, controla repetida, llama a `process_guess()`. Si es False imprime `'That letter is not in the phrase.'`. Luego siempre llama a `display_status()` y vuelve a pedir letra.
Alternativo / Excepcion	Entrada invalida → CU-06. Repetida → CU-06.

CU-03 — Ver progreso

	Jugador mira tras una jugada.

	Sistema imprime `'Word:'` + progreso con espacios + `'Guessed letters: ...'` (orden alfabetico) + `'Attempts remaining: ...'`.
Alternativo / Excepcion	Antes de la primera jugada valida no hay progreso visible.

CU-04 — Ganar partida

	Jugador completa la ultima letra.
	Sistema sale del `while not is_over`, `is_won()` es True e imprime `'¡Ganaste! La palabra era: {word}'`.
Alternativo / Excepcion	No se pide otra letra.

CU-05 — Perder partida

	Jugador comete el 6to error.
	Sistema sale del bucle, `is_lost()` es True e imprime `'Fin del juego. La palabra era: {word}'`.
Alternativo / Excepcion	No se pide otra letra.

CU-06 — Entrada invalida / repetida

	Jugador escribe vacio, numero, simbolo, varias letras, o una letra ya usada.
	Si es invalida → `'Enter a single letter.'` y `continue`. Si es repetida → `'You already guessed that letter.'` y `continue`. En ambos: **no** se llama a `process_guess()`, **no** se llama a `display_status()`, **no** cambia `errors`.
Alternativo / Excepcion	Si se escribe 'AB', se rechaza completo: no se toma solo la primera letra.

CU-07 — Ver letras e intentos

	Jugador quiere saber que ya probo.
	Es parte de `display_status()`: letras ordenadas alfabeticamente + intentos restantes = `max_errors - errors`.
Alternativo / Excepcion	Si no hay letras, `format_guessed_letters` devolveria `'none'`, pero en main eso no se muestra.

Requisitos

ID	Requisito (verificable en codigo)	Tipo
RF-01	Elegir palabra al azar de `wordlist.txt` con `get_random_word()`.	Funcional
RF-02	Aceptar una letra por turno, con `.lower()` en `input`.	Funcional
RF-03	Revelar todas las posiciones de un acierto (via `display_progress`).	Funcional
RF-04	Contar como error solo letras nuevas e incorrectas. Maximo 6.	Funcional
RF-05	Terminar al ganar o al llegar a 6 errores, mostrando la palabra tal cual.	Funcional
RF-06	Rechazar entradas invalidas y repetidas sin penalizar y sin mostrar `display_status`.	Funcional
RF-07	Revelar automaticamente caracteres no alfabeticos y no exigirlos para ganar.	Funcional
RNF-01	Solo Python 3 + biblioteca estandar.	No funcional
RNF-02	Cada turno O(n) donde n = largo de palabra; partida completa trivial.	No funcional

6 Diagramas dinamicos

6.1 Maquina de estados

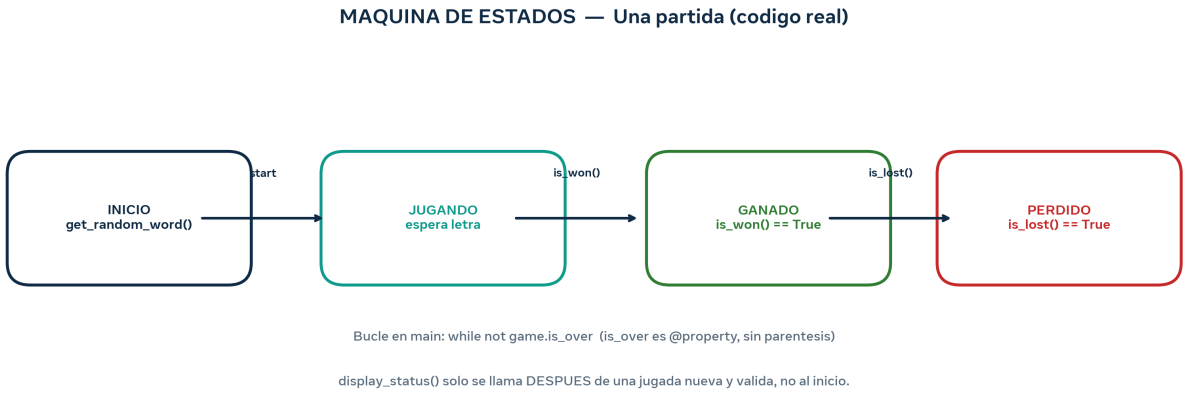


Figura 3 — GANADO y PERDIDO son terminales. La condicion del bucle es la propiedad is_over.

6.2 Diagrama de secuencia — una jugada real

Paso	Quien → Quien	Mensaje real	Que pasa
1	Jugador → main	`python3 main.py`	Inicia.
2	main → utils	`get_random_word()`	Lee y elige.
3	main → game	`HangmanGame(word)`	Crea estado.
4	main → game	`game.is_over` (propiedad)	Si es False, sigue.
5	main → Jugador	`Guess a letter:`	Pide letra (sin mostrar palabra aun la primera vez).
6	Jugador → main	Escribe letra	Entrada.
7	main → utils	`is_valid_guess(guess)`	Si es False → 'Enter a single letter.' + continue.
8	main → game	`guess in guessed_letters ?`	Si es True → 'You already guessed...' + continue.
9	main → game	`process_guess(guess)`	Actualiza estado. Si False → 'That letter is not in the phrase.'
10	main → game/utils	`display_status(game)`	Muestra Word + Guessed + Attempts.
11	main → game	`game.is_over`	Si es True → '¡Ganaste!...' o 'Fin del juego...'

6.3 Flujograma

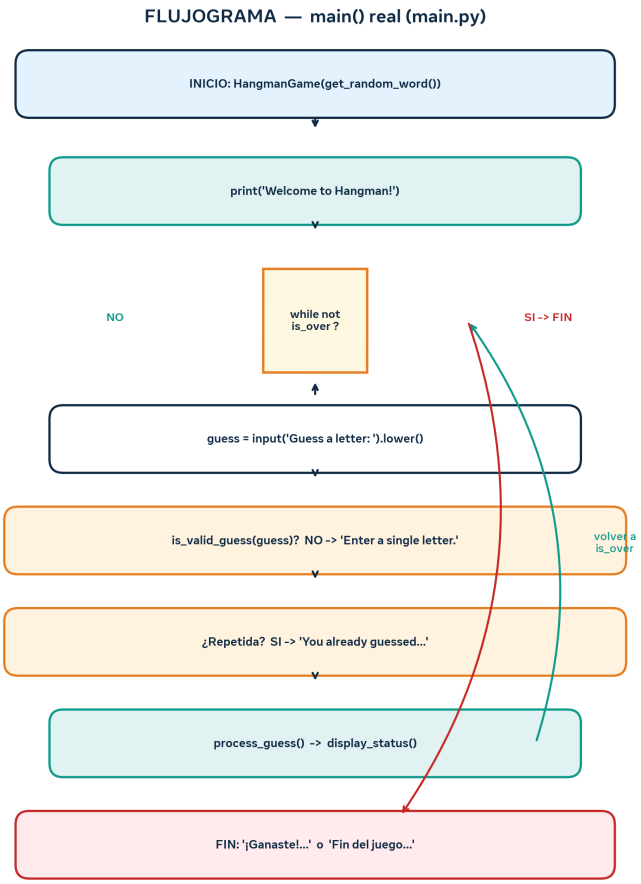


Figura 4 — El bucle real: validar → controlar repetida → process_guess → display_status → volver a is_over.

7 Simulaciones de pantallas — salida real del codigo

Se usa `python` como ejemplo (una de las 15 palabras). Los textos entre comillas son literales del codigo. No hay dibujo de horca ni 'jugar otra vez' en el codigo actual.

Pantalla 1 — Inicio (antes de la primera jugada valida no hay Word)

```
Welcome to Hangman!
Guess a letter:
```

Pantalla 2 — Primer acierto

```
Welcome to Hangman!
Guess a letter: p
Word:
p _ _ _ _ _
Guessed letters: p
Attempts remaining: 6
Guess a letter:
```

Observa: `display_status()` aparece **despues** de la jugada, no antes. Y `Attempts remaining` es `6 - 0 = 6` porque el acierto no resta.

Pantalla 3 — Error

```
Guess a letter: z
That letter is not in the phrase.
Word:
p _ _ _ _ _
Guessed letters: p, z
Attempts remaining: 5
Guess a letter:
```

Nota: `Guessed letters` sale ordenado alfabeticamente (`p, z`), aunque el orden de jugada fue p luego z. Con jugadas z luego a, se mostraria `a, z`.

Pantalla 4 — Entrada invalida y repetida (no cambian nada)

```
Guess a letter: 12
Enter a single letter.
Guess a letter: p
You already guessed that letter.
Guess a letter:
```

En ambos casos no se imprime `Word:` ni `Guessed letters` ni `Attempts`, porque `continue` salta `display_status()`.

Pantalla 5 — Victoria

```
Guess a letter: n
Word:
p y t h o n
Guessed letters: h, n, o, p, t, y, z
Attempts remaining: 5
¡Ganaste! La palabra era: python
```

La ultima jugada muestra el display completo y *luego* el mensaje de victoria. El mensaje final mezcla ingles y espanol tal cual esta en el codigo: `'¡Ganaste! La palabra era: python'`.

Pantalla 6 — Derrota

```
Guess a letter: q
That letter is not in the phrase.
Word:
p _ _ _ _ _
Guessed letters: p, q, x, j, k, w, z
Attempts remaining: 0
Fin del juego. La palabra era: python
```

Al llegar a 6 errores, `is_lost()` es True, `is_over` es True, el bucle termina y se imprime `'Fin del juego. La palabra era: python'`.

Criterio de aceptacion de pantallas (codigo real)

(1) Nunca se muestra la palabra antes de terminar, salvo letra a letra en el progreso. (2) El numero de simbolos en `Word:` siempre coincide con `len(word)`. (3) `Attempts remaining` nunca baja de 0 en el flujo de `main`. (4) Entrada invalida o repetida nunca suma error ni muestra `display_status`.

8 Explicacion de los algoritmos — con codigo real

Se cita el codigo tal cual esta en los archivos, sin simplificar. Cada algoritmo incluye idea, ejemplo verificado por ejecucion y complejidad.

8.1 A1 — Seleccion aleatoria uniforme: get_random_word()

```
def get_random_word():
    wordlist_path = Path(__file__).with_name('wordlist.txt')
    words = [line.strip() for line in wordlist_path.read_text(encoding='utf-8').splitlines
             if line.strip().isalpha()]
    if not words:
        raise ValueError('wordlist.txt must contain at least one alphabetic word')
    return random.choice(words)
```

Idea: leer todo el archivo, filtrar, elegir con `random.choice()`. Cada palabra 1/15. **Por que** `Path(__file__)`**:** la ruta no depende de desde donde se ejecuta `python main.py`. **Complejidad:** leer + filtrar O(N), N=15 lineas; elegir O(1); memoria O(N).

8.2 A2 — Validacion: is_valid_guess()

```
def is_valid_guess(guess):
    return len(guess) == 1 and guess.isalpha()
# main hace antes: guess = input('Guess a letter: ').lower()
```

Tabla real: `'a'` → `True`, `'A'` → `True` (`main` ya la bajo a `'a'`), `''` → `False`, `'ab'` → `False`, `'1'` → `False`, `'-'` → `False`, `' '` → `False`, `' A '` → `False` (`len 3`). **Complejidad:** O(1). **Limite:** no hace `strip()`; si se llama directamente con `None` o un entero, `len()` lanza `TypeError`, no `ValueError`. En `main` siempre llega un `str` de `input()`, asi que no se ve ese caso.

8.3 A3 — Transicion de estado: process_guess() y guess()

```
    def process_guess(self, letter):
        if not is_valid_guess(letter):
            raise ValueError('guess must be a single letter')
        letter = letter.lower()
        if letter in self.guessed_letters:
            return letter in self.word.lower()
        self.guessed_letters.append(letter)
        if letter not in self.word.lower():
            self.errors += 1
            return False
        return True

    def guess(self, letter):
        normalized_letter = letter.lower() if isinstance(letter, str) else letter
        if normalized_letter in self.guessed_letters:
            return None
        return self.process_guess(letter)
```

Diferencia process_guess vs guess, verificada: con `HangmanGame('Python')`, `process_guess('p')` → `True`, `process_guess('P')` → `True` (repetida, sin cambiar `errors`). En cambio `guess('p')` repetida → `None`, `guess('z')` → `False` y `errors=1`, `guess('Z')` repetida → `None` y `errors` sigue en 1. `main` no usa `guess()`: usa `process_guess()` despues de su propio chequeo de repetida.

Ejemplo paso a paso — `word='python'`:

Jugada en main	Que devuelve process_guess	Estado despues
p	True	`guessed=['p'], errors=0`
z	False + mensaje 'That letter...'	`guessed=['p','z'], errors=1`
p repetida	No llega a process_guess (main hace continue)	Sin cambios
o	True	`guessed=['p','z','o'], errors=1`

Complejidad: `letter in guessed_letters` es O(k) con lista (k ≤ 26), `letter in word.lower()` es O(n). Por jugada O(n + k). Con `set` seria O(1) para la primera, pero el ejercicio exige lista.

8.4 A4 — Mostrar progreso: display_progress() vs display_word()

```
    # game.py — imprime con espacios
    def display_progress(self):
        progress = []
        for character in self.word:
            if not character.isalpha(): progress.append(character)
            elif character.lower() in self.guessed_letters: progress.append(character)
            else: progress.append('_')
        print(' '.join(progress))

    # utils.py — devuelve sin espacios, no la usa main
    def display_word(phrase, guessed_letters):
        return ''.join(character if not character.isalpha() or character.lower() in
    guessed_letters else '_'
                       for character in phrase)
```

Verificado: `display_word('python', ['p','o']) == 'p___o_'`; `display_word('hello world', []) == '_____ _____'` (el espacio se revela solo); `HangmanGame('hello world') + process_guess('l')` → `display_progress()` imprime `'_ _ l l _ _ _ _ l _'`. **Por que dos versiones:** una para consola (con espacios, legible) y una pura para pruebas (devuelve cadena). El riesgo es que diverjan: si se cambia una, hay que cambiar la otra.

Complejidad: O(n) tiempo, O(n) memoria para la lista / cadena.

8.5 A5 — Terminacion: is_won(), is_lost(), is_over y el bucle

```
    def is_won(self):
        return all(not c.isalpha() or c.lower() in self.guessed_letters for c in self.word)
    def is_lost(self):
        return self.errors >= self.max_errors
    @property
    def is_over(self):
        return self.is_won() or self.is_lost()

    # main.py
    while not game.is_over:   # <- propiedad, sin ()
        guess = input('Guess a letter: ').lower()
        if not is_valid_guess(guess):
            print('Enter a single letter.'); continue
        if guess in game.guessed_letters:
            print('You already guessed that letter.'); continue
        if not game.process_guess(guess):
            print('That letter is not in the phrase.')
        display_status(game)
```

Terminacion garantizada: cada vuelta que llega a `process_guess()` con letra nueva hace crecer `guessed_letters` en 1 y, si es error, `errors` en 1. Como hay a lo sumo 26 letras distintas, tras a lo sumo 26 jugadas nuevas el juego termina: o se completan las letras distintas de la palabra (victoria) o se llega a 6 errores (derrota). Las entradas invalidas y repetidas usan `continue` sin cambiar estado, pero no impiden una jugada nueva posterior.

Formato de letras: `format_guessed_letters()` hace `', '.join(sorted(...)) or 'none'`: ordenar $k \leq 26$ letras es O(k log k), despreciable.

Resumen de complejidades (codigo real)

Algoritmo	Tiempo	Memoria	Cuando
get_random_word	O(N), N=15	O(N)	Una vez por partida
is_valid_guess	O(1)	O(1)	Por turno
process_guess / guess	O(n + k)	O(1)	Por turno
display_progress / display_word	O(n)	O(n)	Por turno valido
is_won / is_lost / is_over	O(n) / O(1) / O(n)	O(1)	Cada vuelta del while
format_guessed_letters	O(k log k)	O(k)	Por display_status

9 Estructuras de datos e invariantes

Estructura	Donde	Por que esta y no otra
`str` para `word`	HangmanGame	Inmutable, ordenada. Se conserva el caso original; las comparaciones usan `.lower()`.
`list[str]` para `guessed_letters`	HangmanGame	Requisito del ejercicio. Preserva orden de jugada (aunque el display lo ordena). Busqueda O(k), $k \leq 26$, aceptable.
`int` para `errors`, `max_errors`	HangmanGame	Contadores simples. No hace falta guardar lista de errores por separado.
`list[str]` para palabras	utils	Necesaria para `random.choice()` por indice.
Archivo de texto	wordlist.txt	Editable sin programar, versionable con Git.

Errores que este diseno evita (y uno que no)

- Contar como error una repetida: lo evitan `main` + `process_guess` + `guess`.
- Comparar sin normalizar: lo evitan `.lower()` en `input` y en `process_guess`.
- Mostrar la palabra secreta antes de tiempo: `display_progress` solo revela letras adivinadas o caracteres no alfabeticos.
- Error que **no** evita: si se llama a `game.is_over()` con parentesis, falla porque `is_over` es propiedad que devuelve `bool`. Hay que usar `game.is_over` sin parentesis.

10 Pruebas — verificadas por ejecucion del codigo real

Se ejecutaron estas pruebas contra el codigo entregado. Los resultados entre comillas son salida real.

#	Caso	Llamada real	Resultado verificado
P-01	Acierto	`HangmanGame('Python'); process_guess('p')`	True, `guessed=['p'], errors=0`, display `'P _ _ _ _ _'`.
P-02	Repetida en process_guess	`process_guess('P')` tras anterior	True, `errors` sigue 0, sin duplicado.
P-03	Repetida en guess	`guess('p')` tras anterior	`None`.
P-04	Error + repetida error	`guess('z'), guess('Z')`	Primero False, `errors=1`; segundo None, `errors` sigue 1.
P-05	Victoria	`HangmanGame('gato'); guess g,a,t,o`	`is_won()=True, is_over=True`.
P-06	Derrota	`HangmanGame('gato'); z,x,b,q,j,k`	`errors=6, is_lost()=True, is_over=True`.
P-07	Validacion	`is_valid_guess(''), ('ab'), ('1'), ('-'), (' ')`	Todas False. `('a'), ('A')` True.
P-08	Constructor invalido	`HangmanGame(''), HangmanGame('123')`	`ValueError: word must contain at least one letter`.
P-09	process_guess invalido	`process_guess('ab')`	`ValueError: guess must be a single letter`.
P-10	display_word	`display_word('python', ['p','o'])`	`'p___o_'`.
P-11	display_word con espacio	`display_word('hello world', [])`	`'_____ _____'`.
P-12	format_guessed	`format_guessed_letters(['z','a','m'])`	`'a, m, z'`; vacia → `'none'`.
P-13	Frase con espacio	`HangmanGame('hello world'); process_guess('l')`	Display `'_ _ l l _ _ _ _ l _'`, `is_won()=False`.
P-14	display_status	`HangmanGame('python'); process_guess('p'); display_status()`	Imprime `'Word: / p _ _ _ _ _ / Guessed letters: p / Attempts remaining: 6'`.

Prueba manual sugerida (la de PRESENTACION.md)

- 1. Ejecutar `python main.py`.
- 2. Probar entrada vacia, numero y varias letras: deben rechazarse sin consumir intentos.
- 3. Adivinar una letra y observar patron, letras intentadas e intentos restantes.
- 4. Repetir una letra, incluso una incorrecta: debe pedir otra sin alterar el estado.

- 5. Mostrar ambos finales: victoria y derrota (6 errores).

Ejemplo con `pytest` usando la API real:

```
def test_repetida_guess_devuelve_none():
    g = HangmanGame('gato')
    assert g.guess('z') is False
    assert g.errors == 1
    assert g.guess('z') is None
    assert g.errors == 1

def test_is_over_es_propiedad():
    g = HangmanGame('gato')
    assert g.is_over is False  # sin parentesis
    for ch in 'gato':
        g.process_guess(ch)
    assert g.is_over is True
```

11 Limitaciones, riesgos y mejoras

Limitaciones reales (no son inventadas)

- Sin 'jugar otra vez': al terminar hay que volver a ejecutar `python main.py`.
- Sin dibujo de horca ASCII: solo se muestra `Attempts remaining`.
- Mensajes bilingues: prompts y errores en ingles (`'Welcome...', 'Guess a letter:', 'Enter a single letter.'`), final en espanol (`'¡Ganaste!...', 'Fin del juego...'`). Para una entrega, conviene unificar idioma.
- Sin manejo de `EOFError / KeyboardInterrupt`: Ctrl+D o Ctrl+C terminan con traceback.
- Dos funciones no usadas por `main`: `guess()` y `display_word()`. Si no se van a usar, hay que decidir: integrarlas o eliminarlas y documentar por que.
- Dos implementaciones de 'mostrar progreso' que pueden divergir: `display_progress()` (con espacios, imprime) y `display_word()` (sin espacios, devuelve).
- Wordlist pequena: 15 palabras. Para una demo repetible, PRESENTACION.md sugiere dejar temporalmente una sola palabra de prueba y restaurar el archivo.

Riesgos

- **Archivo vacio / sin palabras alfabeticas:** `get_random_word()` lanza `ValueError` sin capturar en `main` → traceback. Mitigacion: capturar en `main` y mostrar mensaje limpio.
- `is_over()` **con parentesis:** error comun para quien lea el codigo como si fuera metodo. Mitigacion: documentarlo (esta seccion) o cambiar a metodo, pero eso romperia `main` actual.
- `is_valid_guess(None)`: `TypeError`, no `ValueError`. En `main` no ocurre, pero si se usa la clase directamente con un tipo no-`str`, conviene validar tipo.

Mejoras futuras (en orden, segun PRESENTACION.md)

#	Mejora	Que toca
1	Opcion 'jugar otra vez' al terminar (bucle exterior + confirmacion).	`main.py`
2	Grafico ASCII de horca por etapas (0 a 6 errores).	`utils.py` + `display_status`
3	Dificultad (facil / normal / dificil) cambiando `max_errors` o filtrando por largo.	`game.py` + `utils.py`
4	Unificar idioma de mensajes y capturar `EOFError / KeyboardInterrupt`.	`main.py`
5	Unificar `guess()` / `process_guess()` y `display_word()` / `display_progress()`, o eliminar la variante no usada.	`game.py` + `utils.py`
6	Puntuaciones persistentes.	Nuevo archivo / modulo

Anexo A Presentacion de 5 minutos (segun PRESENTACION.md)

Guion ya entregado en `PRESENTACION.md`. Resumen para el equipo:

Tiempo	Quien / Que	Contenido
0:00-0:35	Integrante 1 — Introduccion	Hangman, consola, palabra al azar, hasta 6 errores. Aplica funciones, clases, modulos, validacion y lectura de archivos.
0:35-1:25	Integrante 1 — Componentes	`main.py` entrada, `game.py` HangmanGame, `utils.py` azar/validacion/formato, `wordlist.txt` una palabra por linea.
1:25-2:20	Integrante 2 — Decisiones	Separacion de responsabilidades, `guessed_letters` lista, `errors` solo con letra incorrecta nueva, `is_won / is_lost`, espacios/signos revelados solos.
2:20-4:20	Integrante 3 — Demo en vivo	`python main.py`: entrada invalida sin consumir intentos, acierto, repetida sin alterar estado, victoria y derrota.
4:20-5:00	Integrantes 1 y 2 — Cierre	Cumple objetivo, separacion permite cambiar lista o reglas. Mejoras opcionales: dificultad, grafico ASCII, puntajes, jugar otra ronda.

Preguntas probables (respuestas cortas, con codigo real)

- **¿Por que separar en modulos?** Para distinguir entrada, reglas/estado, auxiliares y datos.
- **¿Como se evita gastar intentos con entrada invalida o repetida?** `main` valida con `is_valid_guess` y comprueba `guessed_letters` antes de `process_guess`.
- **¿Que pasa con espacios y signos?** `display_progress` los revela y `is_won` solo exige letras alfabeticas.
- **¿Como se determina la derrota?** Cada letra nueva que no esta en la palabra sube `errors`; derrota cuando llega a `max_errors = 6`.
- **¿Que cambiarian para otra ronda?** Envolver creacion y ejecucion en un ciclo exterior y pedir confirmacion al terminar.

Anexo B Checklist de entrega

Item	Verificado
Codigo en `hangman_project/` con los 6 archivos (`main`, `game`, `utils`, `wordlist`, `README`, `PRESENTACION`).	■
`python3 main.py` inicia sin errores.	■
Se puede ganar y se puede perder (6 errores).	■
Entrada invalida y repetida no cuentan como error ni muestran `display_status`.	■
Esta v2 (no la v1) acompana la entrega.	■
Se probo la demo con una palabra de prueba y se restauro `wordlist.txt` a las 15 palabras.	■

Fuentes y metodo

Archivos verificados y ejecutados: `main.py`, `game.py`, `utils.py`, `wordlist.txt`, `README.md`, `PRESENTACION.md`. Se ejecutaron pruebas reales (ver Seccion 10) para confirmar: `guess()` → `None` en repetidas, `process_guess()` en repetidas, `display_word()`, `format_guessed_letters()`, victoria, derrota, constructores invalidos y progreso con espacios. Repositorio: **github.[STRIPPED 68 bytes]**.

— Fin del documento — v2 corregida —